

Python for Computer Science and Data Science 2 (CSE 3652)

MAJOR ASSIGNMENT-3: MACHINE LEARNING- CLASSIFICATION, REGRESSION AND CLUSTERING

Overview

Imagine you've been hired by *FashionX*, a fast-growing online fashion retailer struggling to organize its expanding product catalog. The company needs an automated system to classify fashion items into categories like T-shirts, Dresses, Shoes, etc., based on images.

As a data scientist, your task is to develop a machine learning model to classify 28x28 pixel images from the *Fashion MNIST* dataset into 10 fashion categories. You'll apply algorithms like K-Nearest Neighbors (KNN) and Support Vector Machines (SVM) for classification. Additionally, you'll use t-SNE to visualize the data, uncovering patterns and clusters in the high-dimensional space.

This project will help FashionX scale its operations by efficiently automating product categorization, making it easier for customers to find what they're looking for.

Tasks

Task 1: Data Preprocessing

- Download the Fashion MNIST dataset using TensorFlow. You can load it directly using the following code:

```
import tensorflow as tf
from tensorflow.keras.datasets import fashion_mnist

# Load the dataset
(train_images, train_labels), (test_images, test_labels) = \
fashion_mnist.load_data()
```

- The dataset consists of 60,000 training images and 10,000 test images of fashion products, with 10 distinct categories. Each image is 28x28 pixels.
- Perform the following preprocessing steps:
 1. Normalize the images (values should be between 0 and 1).
 2. Flatten the 28x28 pixel images into 1D arrays (784 pixels per image).
 3. Handle any missing or incorrect values (if any).
- Split the dataset into training and test sets.

```
In [ ]: # =====
# TASK 1: DATA PREPROCESSING
# =====

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from tensorflow.keras.datasets import fashion_mnist

# Load dataset
(train_images, train_labels), (test_images, test_labels) = fashion_mnist.load_data()

# Display dataset shape
print("Training Data Shape:", train_images.shape)
```

```
print("Testing Data Shape:", test_images.shape)

# Normalize pixel values
train_images = train_images / 255.0
test_images = test_images / 255.0

# Flatten images from 28x28 to 784
train_images_flat = train_images.reshape((60000, 784))
test_images_flat = test_images.reshape((10000, 784))

print("Flattened Training Shape:", train_images_flat.shape)
print("Flattened Testing Shape:", test_images_flat.shape)

# Check missing values
print("Missing values in training set:", np.isnan(train_images_flat).sum())
print("Missing values in testing set:", np.isnan(test_images_flat).sum())

# Display sample images
class_names=['T-shirt/top', 'Trouser', 'Pullover', 'Dress',
             'Coat', 'Sandal', 'Shirt', 'Sneaker',
             'Bag', 'Ankle boot']

plt.figure(figsize=(10,10))

for i in range(9):
    plt.subplot(3,3,i+1)
    plt.imshow(train_images[i], cmap='gray')
    plt.title(class_names[train_labels[i]])
    plt.axis('off')

plt.show()
```

Training Data Shape: (60000, 28, 28)
Testing Data Shape: (10000, 28, 28)
Flattened Training Shape: (60000, 784)
Flattened Testing Shape: (10000, 784)
Missing values in training set: 0
Missing values in testing set: 0



Task 2: K-Nearest Neighbors (KNN) Classification

- Implement the K-Nearest Neighbors (KNN) algorithm using the preprocessed dataset.
- Experiment with different values of k (e.g., $k = 3$, $k = 5$, $k = 7$) to see how the model performance changes.
- Evaluate the model performance using accuracy on the test dataset.
- Provide a comparison of different k values and the impact on accuracy.

```
In [ ]: # =====  
# TASK 2: KNN CLASSIFICATION  
# =====  
  
import numpy as np  
from tensorflow.keras.datasets import fashion_mnist  
from sklearn.neighbors import KNeighborsClassifier  
from sklearn.metrics import accuracy_score  
  
# Load dataset  
(train_images, train_labels), (test_images, test_labels) = fashion_mnist.load_data()  
# Normalize images
```

```
train_images = train_images / 255.0
test_images = test_images / 255.0

# Flatten images
train_images_flat = train_images.reshape((60000, 784))
test_images_flat = test_images.reshape((10000, 784))

# Different k values
k_values = [3, 5, 7]

# Train and evaluate KNN
for k in k_values:

    print(f"\nTraining KNN with k = {k}")

    # Create KNN model
    knn = KNeighborsClassifier(n_neighbors=k)

    # Train model
    knn.fit(train_images_flat, train_labels)

    # Predict test data
    predictions = knn.predict(test_images_flat)

    # Calculate accuracy
    accuracy_score(test_labels, predictions)
    print(f"Accuracy for k={k}: {accuracy:.4f}")
```

Training KNN with k = 3
Accuracy for k=3: 0.8541

Training KNN with k = 5
Accuracy for k=5: 0.8554

Training KNN with k = 7
Accuracy for k=7: 0.8540

Task 3: Support Vector Machine (SVM) Classification

- Train a Support Vector Machine (SVM) classifier on the same preprocessed dataset.
- Experiment with different kernels (linear, polynomial, radial basis function) and hyperparameters such as C .
- Evaluate the model performance using accuracy on the test dataset.
- Compare the SVM performance with that of the KNN model.

```
In [ ]: # =====
# TASK 3: SVM CLASSIFICATION
# =====

import numpy as np
from tensorflow.keras.datasets import fashion_mnist
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# Load dataset
(train_images, train_labels), (test_images, test_labels) = fashion_mnist.load_data()
# Normalize images
train_images = train_images / 255.0
test_images = test_images / 255.0
# Flatten images
train_images_flat = train_images.reshape((60000, 784))
test_images_flat = test_images.reshape((10000, 784))
```

```
# Kernels to test
kernels = ['linear', 'poly', 'rbf']

# Train and evaluate SVM
for kernel in kernels:

    print(f"\nTraining SVM with {kernel} kernel")

    # Create SVM model
    svm = SVC(kernel=kernel, C=1)

    # Train model
    svm.fit(train_images_flat, train_labels)

    # Predict test data
    predictions = svm.predict(test_images_flat)

    # Calculate accuracy
    accuracy = accuracy_score(test_labels, predictions)

    print(f"Accuracy using {kernel} kernel: {accuracy:.4f}")
```

Training SVM with linear kernel
Accuracy using linear kernel: 0.8464

Training SVM with poly kernel
Accuracy using poly kernel: 0.8630

Training SVM with rbf kernel
Accuracy using rbf kernel: 0.8828

Task 4: Data Visualization with t-SNE

- Use the t-SNE technique to reduce the dimensionality of the data from 784 features to 2 or 3 dimensions.
- Visualize the 2D or 3D representation of the Fashion MNIST dataset and observe the clustering of different fashion categories.
- Analyze the plot to identify how well the categories are separated, and discuss the results.

```
In [ ]: # =====
# TASK 4: t-SNE VISUALIZATION
# =====

import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import fashion_mnist
from sklearn.manifold import TSNE

# Load dataset
(train_images, train_labels), (test_images, test_labels) = fashion_mnist.load_data()
# Normalize images
train_images = train_images / 255.0
# Flatten images
train_images_flat = train_images.reshape((60000, 784))

# Use subset for faster processing
sample_data = train_images_flat[:5000]
sample_labels = train_labels[:5000]

# Apply t-SNE
tsne = TSNE(n_components=2, random_state=42)

reduced_data = tsne.fit_transform(sample_data)
```

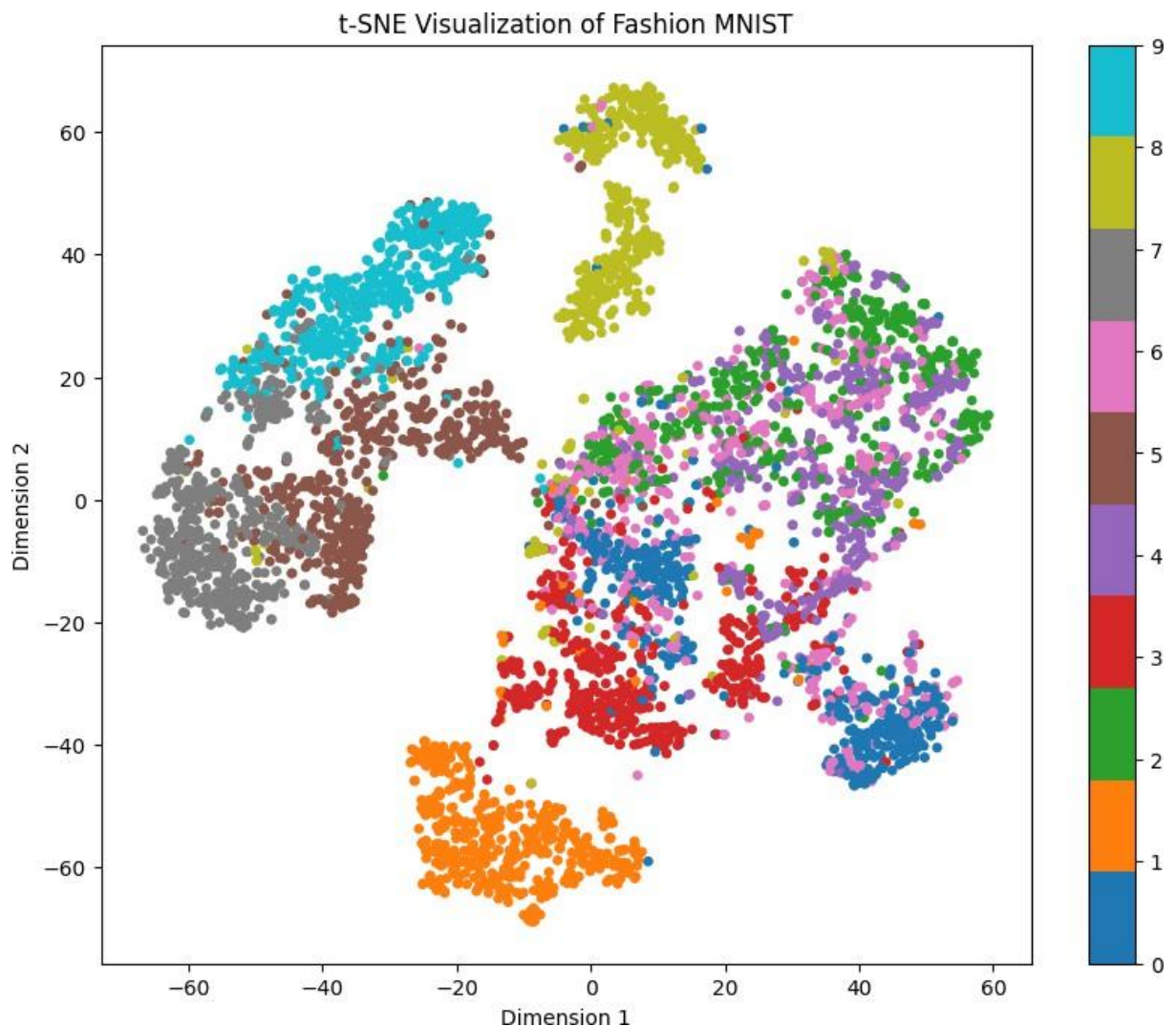
```
# Plot visualization
plt.figure(figsize=(10,8))

scatter = plt.scatter(
    reduced_data[:,0],
    reduced_data[:,1],
    c=sample_labels,
    cmap='tab10',
    s=15
)

plt.colorbar(scatter)

plt.title('t-SNE Visualization of Fashion MNIST')
plt.xlabel('Dimension 1')
plt.ylabel('Dimension 2')

plt.show()
```



Task 5: Model Evaluation and Reporting

- Evaluate the performance of both the KNN and SVM models using accuracy, precision, recall, F1-score, and confusion matrix.
- Discuss which model performs better and why, based on the evaluation metrics.
- Write a report summarizing the approach used in each task, the results obtained, and insights derived from the visualizations.

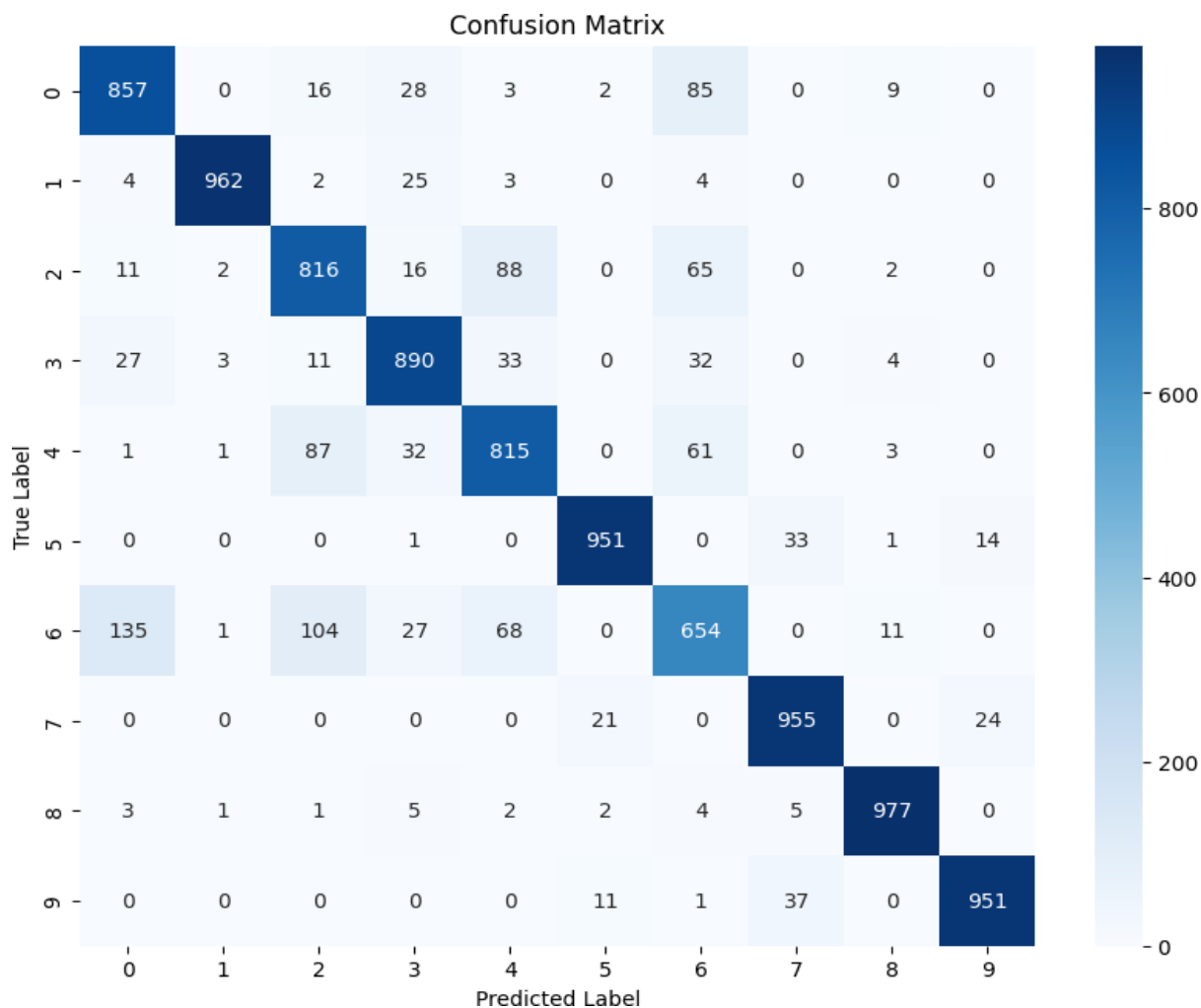
In []:

```
# =====  
# TASK 5: MODEL EVALUATION  
# =====  
  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns  
  
from tensorflow.keras.datasets import fashion_mnist  
from sklearn.svm import SVC  
from sklearn.metrics import accuracy_score  
from sklearn.metrics import classification_report  
from sklearn.metrics import confusion_matrix  
  
# Load dataset  
(train_images, train_labels), (test_images, test_labels) = fashion_mnist.load_data()  
  
# Normalize images  
train_images = train_images / 255.0  
test_images = test_images / 255.0  
  
# Flatten images  
train_images_flat = train_images.reshape((60000, 784))  
test_images_flat = test_images.reshape((10000, 784))  
  
# Best model: SVM with RBF kernel  
svm = SVC(kernel='rbf', C=1)  
  
# Train model  
svm.fit(train_images_flat, train_labels)  
  
# Predict test data  
predictions = svm.predict(test_images_flat)  
  
# Accuracy  
accuracy = accuracy_score(test_labels, predictions)  
  
print("\nModel Accuracy:", accuracy)  
  
# Classification Report  
print("\nClassification Report:")  
print(classification_report(test_labels, predictions))  
  
# Confusion Matrix  
cm = confusion_matrix(test_labels, predictions)  
  
# Plot confusion matrix  
plt.figure(figsize=(10,8))  
  
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')  
  
plt.xlabel('Predicted Label')  
plt.ylabel('True Label')  
plt.title('Confusion Matrix')  
  
plt.show()
```


Model Accuracy: 0.8828

Classification Report:

	precision	recall	f1-score	support
0	0.83	0.86	0.84	1000
1	0.99	0.96	0.98	1000
2	0.79	0.82	0.80	1000
3	0.87	0.89	0.88	1000
4	0.81	0.81	0.81	1000
5	0.96	0.95	0.96	1000
6	0.72	0.65	0.69	1000
7	0.93	0.95	0.94	1000
8	0.97	0.98	0.97	1000
9	0.96	0.95	0.96	1000
accuracy			0.88	10000
macro avg	0.88	0.88	0.88	10000
weighted avg	0.88	0.88	0.88	10000



Submitted By-

- Name- Amulya Shrivastava
- Regd No- 2341019456
- Section- 23412F3
- Serial No- 59
- Subject- PFCD2 Major Assignment-03